Collision-Conditioned Reasoning and Planning for Autonomous Manipulation from Partial View Point Clouds

Ashton Larkin[1] and Tucker Hermans[1,2]

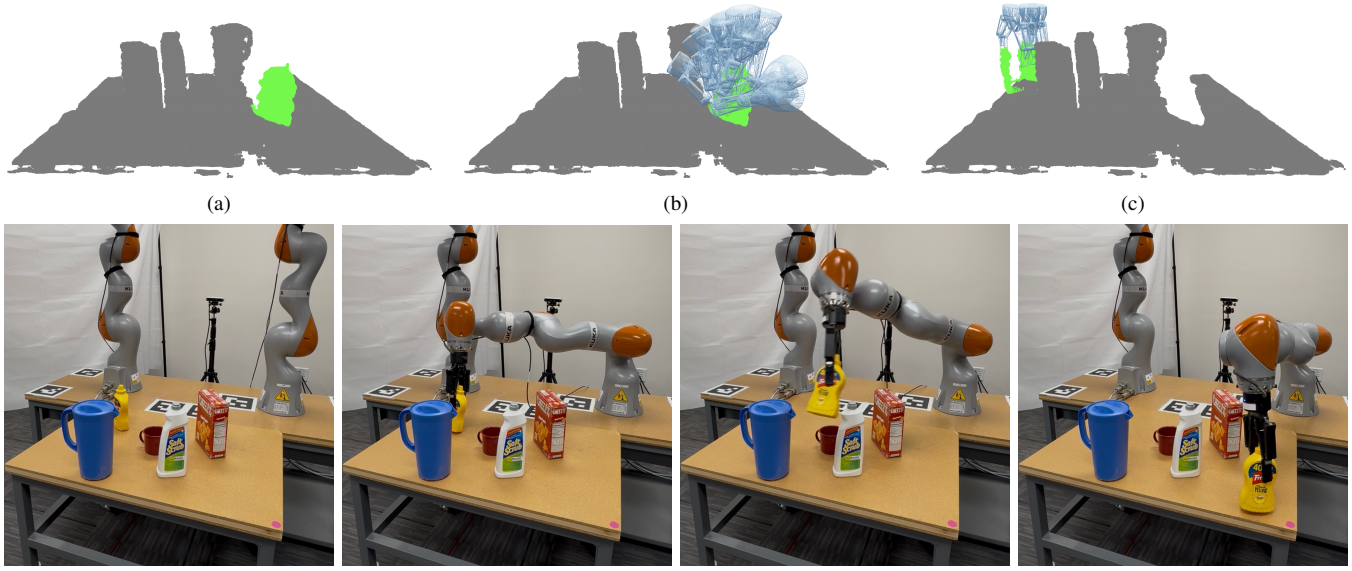


Fig. 1. A successful hardware trial in configuration 1. Top: the planner's view of the initial scene, seen from the depth camera on the opposite side of the table from the photos below, so the scene appears mirrored. It shows (a) the segmented point cloud, (b) the successfully planned grasps on the mustard bottle, and (c) the mustard bottle point cloud at the successfully planned placement poses, with the corresponding grasps. Bottom: from left to right, the robot picks the mustard bottle from the clutter and places it near the desired placement location T_p^{des}, the table corner marked by the pink sticker.

***Abstract*— Placing an object in a cluttered scene requires choosing a grasp, a placement pose and a collision-free trajectory that work together. Prior work shows that jointly reasoning about pick and place is essential for place success, but relies on hand-defined placement costs and slow optimization. Learned collision models can screen candidate placements from partial-view point clouds, but existing pick and place systems that use them commit to a grasp before searching for a placement. We present a collision network that predicts, from partial-view point clouds of the object and environment, whether the object collides with the environment at a candidate placement pose. We combine this network with a learned grasp generator and a GPU-vectorized motion planner to form a collision-conditioned pick and place planning framework that plans for many grasp and placement pairs in parallel. In hardware experiments, our approach is roughly 3.5× faster than a joint pick and place optimization baseline.**

I. INTRODUCTION

Placing objects in clutter is a fundamental requirement of autonomous manipulation. Devendran Shanthi and Hermans [1] show that to ensure successful object placement, pick and place must be viewed as a joint procedure instead of two sequential tasks. They formulate the joint problem as a constrained optimization, but use simple heuristics to evaluate placement quality. Their gradient-based solver is also sensitive to initialization and time-consuming [2], and the trajectory between the optimized pick and place configurations is planned only afterward, so a solution may admit no collision-free trajectory.

A natural alternative to hand-defined placement costs is a learned model of placement quality. Stable placement has been studied extensively, but existing methods either do not take motion planning into account or are too slow to use in it. Nadeau and Kelly [3] sample contact points from a "static robustness map" to derive stable placements, but require object models, which are often unavailable. Nadeau et al. [4] train a diffusion model on labels from this approach to predict placement poses from scene and object point clouds, and Noh et al. [5] learn object stability from physics-simulated data, but neither integrates these models with motion planning, so a predicted placement may not be reachable by the robot. Wingender et al. [6] do use a model-free placeability metric for motion planning, but their Monte Carlo sampling incurs long runtimes that preclude real-time planning.

Learned collision models address a complementary ques-

This material is based upon work supported by the National Science Foundation CISE Graduate Fellowships under Grant No. 2313998. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

[1]Kahlert School of Computing and the Robotics Center, University of Utah, Salt Lake City, UT 84112 USA ashton.larkin@utah.edu
[2]NVIDIA, Seattle, WA 95050 USA

tion: whether the object fits at a pose without hitting its surroundings. When object models are unavailable, a geometric collision check against a partial-view point cloud misses occluded geometry. SceneCollisionNet [7] and CabiNet [8] instead learn to predict object–scene collisions directly from point clouds, use the learned model to filter sampled placements, and plan collision-free trajectories with model predictive path integral control. However, both commit to a grasp before searching for placements for the object in hand, the sequential pick-then-place strategy that [1] show can fail.

Our planning approach builds on work in grasping. Matak et al. [2] plan to many generated grasps in parallel, then evaluate each grasp where its trajectory actually ends instead of at the planner's target. This improves grasp success over the traditional generator–evaluator–planner pipeline, but they address grasping only and do not reason about placement.

Our work combines the joint pick and place reasoning of [1] with learned collision prediction from partial-view point clouds [7], [8], and extends the idea of [2] from grasping to pick and place. We train a collision network that takes partial-view point clouds of the target object and environment and predicts whether the object collides with the environment at a candidate placement pose. Because it scores all candidates in a single forward pass, the network screens placements cheaply before any grasp-specific planning. Together with a learned grasp model and a vectorized motion planner, this lets us plan trajectories for many grasp and placement pairs in parallel. We see this as one step toward an efficient, unified framework for placement reasoning and motion planning. Extending the network from collision to stability reasoning [3]–[6] is planned future work.

We model the collision-conditioned pick and place problem as a constrained optimization as in [1]. In contrast to [1], we include the trajectory from grasp to placement in the optimization instead of planning it afterward. In hardware experiments, our approach is roughly 3.5× faster than [1].

II. PROBLEM FORMULATION

Our goal is to execute a motion plan that satisfies collision-free placement of an object O in a cluttered environment E. We follow the problem formulation from [1], formulating this problem as pick and place, which can be solved by constrained optimization.

We define a pick configuration $\boldsymbol{\theta}_g$ to be an arm configuration that satisfies an end-effector grasp pose for O, defined as $T_g^O \in SE(3)$. We can then define the probability of successfully grasping the object as:

$$P(r_g = 1 | Z_O, T_g^O, \boldsymbol{\theta}_g). \tag{1}$$

In order to place O in E, we must find a collision-free placement pose $T_p \in SE(3)$ for O along with a corresponding placement configuration $\boldsymbol{\theta}_p$. We assume that O is placed in a stable resting orientation on the placement surface, so T_p is restricted to resting poses and placement success depends on whether O collides with other objects in E. The probability that O is collision-free after being placed at T_p in E can be defined as:

$$P(r_p = 1 \mid Z_O, Z_E, T_p, T_g^O, \boldsymbol{\theta}_p). \tag{2}$$

In equations (1) and (2), the object and environment are approximated by partial view point clouds Z_O and Z_E respectively. The joint probability for pick and collision-free placement success is then the product of equations (1) and (2).

As shown in [1], joint inference over pick and place parameters is required. We define the pick and place inference problem as finding $\Phi = (T_g^O, T_p, \boldsymbol{\theta}_g, \boldsymbol{\theta}_p, \tau)$, the tuple of object grasp, placement pose, pick configuration, place configuration, and trajectory that maximizes the joint probability.

Maximizing the joint probability is equivalent to minimizing the negative log joint probability. We use this as the objective in the following constrained optimization problem:

$$\begin{aligned}
\arg\min_{\Phi} \; & -\ln P(r_g = 1 | Z_O, T_g^O, \boldsymbol{\theta}_g) && \text{(3a)}\\
& -\ln P(r_p = 1 \mid Z_O, Z_E, T_p, T_g^O, \boldsymbol{\theta}_p) \\
\text{subject to } & T_g^O \in \mathcal{G}_f && \text{(3b)}\\
& T_p \in \mathcal{P} && \text{(3c)}\\
& \phi(\boldsymbol{\theta}_g) = T_O\, T_g^O; \quad \phi(\boldsymbol{\theta}_p) = T_p\, T_g^O && \text{(3d)}\\
& \boldsymbol{\theta}^- \le \tau(s) \le \boldsymbol{\theta}^+ && \text{(3e)}\\
& \tau(0) = \boldsymbol{\theta}_g, \quad \tau(1) = \boldsymbol{\theta}_p && \text{(3f)}\\
& \epsilon \le \mathrm{SDF}\big(\mathcal{B}(s), Z_E\big) \quad \forall\, s \in [0,1]. && \text{(3g)}
\end{aligned}$$

Equation (3b) requires an object grasp to be one from a set of feasible grasps $\mathcal{G}_f$. Equation (3c) constrains the placement pose T_p to be within the allowed placement region $\mathcal{P}$. $\phi(\boldsymbol{\theta})$ encodes the arm forward kinematics for configuration $\boldsymbol{\theta}$, giving the end-effector pose in the world frame. Equation (3d) requires the end-effector pose to equal $T_O T_g^O$ at the pick configuration $\boldsymbol{\theta}_g$ and $T_p T_g^O$ at the place configuration $\boldsymbol{\theta}_p$, where T_O is the object's initial pose.

For a joint-space trajectory $\tau : [0,1] \to \mathcal{Q}$, where $\mathcal{Q}$ is the robot joint space, equation (3f) sets the endpoints of the trajectory to the pick configuration and the place configuration. Equation (3e) ensures that all configurations along the trajectory are within the robot joint limits.

$R(\boldsymbol{\theta})$ is defined as the robot geometry (arm and hand) at joint configuration $\boldsymbol{\theta}$, expressed in the world frame via forward kinematics. We assume the hand rigidly holds the object during transport, allowing us to define the object pose along the trajectory as $T_O(s) = \phi(\tau(s))\,(T_g^O)^{-1}$, which satisfies $T_O(0) = T_O$ and $T_O(1) = T_p$. The body $\mathcal{B}(s) = R(\tau(s)) \cup T_O(s)\, Z_O$ is the union of the robot geometry $R(\cdot)$ and the held object. Equation (3g) encodes the collision constraint of the trajectory, requiring $\mathcal{B}(s)$ to stay at least ϵ distance from the environment Z_E for all s.

III. APPROACH

We make two primary contributions over previous work in tackling the joint pick-and-place planning problem. First

we propose using learned grasp and placement models that take partial view point clouds as input, as opposed to a heuristic placement evaluator used in [1]. We train a point cloud collision network for placement, while we rely on a pretrained grasp model. We combine these two models with a vectorized motion planner. This allows us to efficiently plan and reason over many grasp candidates in parallel, significantly decreasing the planning time relative to previous work. We now present the details of our placement and grasp models, as well as how we combine them into a unified algorithm to approximately solve problem (3).

A. Collision Model

We approximate the collision-free placement probability in (2) with a neural network G_ψ. The network takes the object point cloud, placed at a candidate pose, and the environment point cloud, and outputs the probability that the object is collision-free at that pose:

$$P(r_p = 1 \mid Z_O, Z_E, T_p, T_g^O, \boldsymbol{\theta}_p) \approx G_\psi(T_p Z_O,\, Z_E)\,. \quad (4)$$

1) Why a Learned Collision Model: The planner already enforces collision avoidance against Z_E in (3g). However, we assume no prior knowledge of the objects in the environment, so Z_E is a partial-view point cloud that omits occluded surfaces, and a geometric check against it can accept placements that intersect hidden parts of the clutter. Following [7], [8], we instead learn to predict collisions from partial-view point clouds, with labels computed from the full object geometry in simulation. The network also scores all candidate placements in a single forward pass, before any grasp-specific planning.

2) Collision-Free Placement Definition: We say that a candidate placement pose T_p is *collision-free* if O, resting on the placement surface at T_p, does not intersect any other object in the environment. Candidate poses are drawn from the placement region $\mathcal{P}$ as resting poses on the surface (a planar position and a yaw about the surface normal), so the object is assumed to be in a stable resting orientation and the network only has to predict collisions with the clutter. A candidate is accepted when $G_\psi(T_p Z_O, Z_E) \geq \delta$.

3) Network Architecture: The network consists of a Point Transformer V3 (PTv3) backbone [9] followed by an MLP classification head. The same backbone, with shared weights, encodes both inputs. Each input is a point cloud of 512 points represented by its 3D coordinates: the environment Z_E and the object $T_p Z_O$. Before encoding, we subtract the centroid of Z_E from both clouds, so that they share a common frame centered on the environment and their relative placement is preserved. The backbone maps each cloud to a single global feature vector, $\mathbf{f}_E, \mathbf{f}_O \in \mathbb{R}^d$ with $d = 512$. We concatenate the two vectors, $\mathbf{f} = [\mathbf{f}_E; \mathbf{f}_O] \in \mathbb{R}^{2d}$, so that the head can reason about the relationship between the object and its surroundings, and pass $\mathbf{f}$ through an MLP with two hidden layers of 128 units with ReLU activations and a single output unit:

$$G_\psi(T_p Z_O, Z_E) = \mathrm{sigmoid}\big(\mathrm{MLP}\big([\mathbf{f}_E; \mathbf{f}_O]\big)\big)\,, \quad (5)$$

where $\mathrm{sigmoid}(\cdot)$ is the logistic function, so the output is the predicted probability that the object is collision-free.

4) Training Data Generation: We generate training data in Isaac Sim [10] using five objects from the YCB dataset [11]. Each object in turn is designated the *target* and the remaining four act as *clutter*. For each target we repeat the following procedure for $N_c = 100$ clutter configurations.

1) **Clutter sampling.** We sample a pose for every clutter object: a uniformly random (x, y) position within the table footprint and a uniformly random yaw about the world z-axis. The height is set so that the lowest vertex of the object's collision mesh lies on the tabletop, and poses with any collision-mesh vertex outside the table are rejected. We accept the configuration only if no two clutter objects overlap, so that every environment is itself physically valid.
2) **Target sampling and labeling.** With the clutter held fixed, we sample target poses using the same tabletop-resting rule until we have collected 50 positive and 50 negative examples. A pose is labeled positive if the simulator reports no overlap between the target and any clutter object, and negative otherwise. Contact with the table is ignored because every sampled pose is a resting pose.
3) **Point clouds.** For each sampled pose we render segmented point clouds and store the target cloud $T_p Z_O$ and the environment cloud Z_E (the clutter and visible tabletop) in the world frame, cropped around the placement region. Samples in which the target is fully occluded are discarded.

Each target therefore yields up to $2 \times 100 \times 50 = 10{,}000$ balanced examples, or 50,000 across the five targets. After discarding occluded samples (about 2.6%), the final dataset contains 48,716 examples. Of the final dataset, 50.68% of the samples are positive and 49.32% of the samples are negative. Since every object serves as both target and clutter in different scenes, and clutter positions and orientations are randomized, the network sees each object in many different environment arrangements.

5) Network Training: The stored point clouds are kept at full resolution. Each time a training example is loaded, we first perturb every coordinate of every point with independent zero-mean Gaussian noise ($\sigma = 0.01$ m, clipped to ± 0.005 m), applied independently to the object and environment clouds, and then randomly downsample each cloud to 512 points (sampling with replacement if it has fewer). Both operations are redrawn at every iteration, so the network sees a different noisy subset of the same scene in each epoch, which acts as data augmentation. Validation and test samples are not jittered, and their 512-point subsamples are drawn with a fixed seed so that metrics are comparable across epochs. We combine the data from all five targets and split it into training, validation and test sets (80/10/10%, stratified by class). We implement the network in PyTorch [12] and minimize the binary cross-entropy loss on the network's pre-sigmoid output with AdamW [13] (learning rate 2×10^{-4}, batch size 64) for at most 100 epochs, stopping after 10 epochs without improvement in validation loss, and keep the

weights with the lowest validation loss.

B. Algorithm

Algorithm 1 shows how we use the pre-trained grasp generator, collision network, and vectorized motion planner to generate and reason over pick and place trajectories from a segmented point cloud. We give greater detail about some of the subroutines below.

1) Grasp Generation and Filtering: In order to approximate the pick probability in equation (1), we choose T_g^O from a finite set of candidates generated from the object point cloud Z_O. We generate candidates with GraspGen [14], an object-centric framework for 6-DOF grasp generation. It combines a diffusion model that samples grasp poses in SE(3) conditioned on the object point cloud with a discriminator that scores each grasp for success; the discriminator is trained on grasps sampled from the generator itself [14]. Let $\mathcal{G} = \{T_{g,i}^O\}_{i=1}^{N_g}$ be the N_g highest-scoring sampled grasps, with discriminator scores $c_i \in [0,1]$. We treat each c_i as an approximate confidence in grasp success, not as a calibrated probability, and use it only to rank grasps.

GraspGen evaluates grasps for the object alone with a free-floating gripper, so $\mathcal{G}$ can contain grasps that the arm cannot reach or that collide with the environment. As in [14], we filter these with the motion planner: we attempt to plan a collision-free trajectory τ_{pick} to each $T_{g,i}^O \in \mathcal{G}$ with cuRobo from the robot's current configuration $\boldsymbol{\theta}_0$, and keep the grasp and its corresponding pick trajectory only if a solution exists. The grasp configuration $\boldsymbol{\theta}_g$ is encoded as the last configuration of τ_{pick}. The set of feasible grasps is

$$\mathcal{G}_f = \big\{T_g^O \in \mathcal{G} \;\big|\; \exists\, \boldsymbol{\theta}_g \in [\boldsymbol{\theta}^-, \boldsymbol{\theta}^+] :\; \phi(\boldsymbol{\theta}_g) = T_O\, T_g^O, \\ \epsilon \le \text{SDF}\big(R(\boldsymbol{\theta}_g), Z_E\big)\big\}, \tag{6}$$

which is needed for (3b).

2) Placement Pose Generation and Collision Filtering: We generate candidate placement poses for O by sampling N_p poses uniformly from the placement region,

$$T_{p,j} \sim \mathcal{U}(\mathcal{P}), \quad j = 1, \ldots, N_p, \tag{7}$$

which ensures that $T_p \in \mathcal{P}$ in (3c). The pose sampling procedure matches what was used during data generation: a uniformly random (x, y) position within the table footprint and a uniformly random yaw about the world z-axis. For each sample, we use $G_\psi(\cdot)$ to evaluate the collision-free placement probability of (2). We keep the candidates whose predicted collision-free probability meets or exceeds a threshold δ,

$$\mathcal{P}_c = \{T_{p,j} \mid G_\psi(T_{p,j} Z_O,\, Z_E) \ge \delta\}, \tag{8}$$

and treat $\mathcal{P}_c$ as the set of candidate placement poses for the optimization. We use $\delta = 0.6$ in all experiments, which trades a small number of rejected valid placements for fewer accepted colliding ones (Section IV-A).

3) Constraint Satisfaction with cuRobo: Given a robot model, the initial object pose T_O, the grasp T_g^O and the placement pose T_p, we use cuRobo [15] to find $\boldsymbol{\theta}_g$, $\boldsymbol{\theta}_p$ and τ satisfying (3d)–(3g). cuRobo discretizes the trajectory into N_t timesteps, $\tau(t/N_t) = \boldsymbol{\theta}_t$, and treats each constraint as a cost term with a large weight, minimizing the sum with a batched L-BFGS solver seeded from many parallel initializations [15].

4) Confidence-Tiered Grasp Search: Given the predicted collision-free placements $\mathcal{P}_c$ and reachable grasps $\mathcal{G}_f$ from Algorithm 1, we seek collision-free trajectories for pairs $(T_g^O, T_p) \in \mathcal{G}_f \times \mathcal{P}_c$. For accurate collision avoidance, the object must be attached to the robot at the grasp T_g^O as in (3g), so each grasp requires its own planning problem and planning every pair is expensive. Following [2], we use cuRobo's batch planning: for a fixed grasp, one call plans to all placements in the current minibatch $\mathcal{P}_m \subseteq \mathcal{P}_c$.

Confidence-Tiered Grasp Search (Algorithm 2) sorts $\mathcal{G}_f$ by grasp confidence and partitions it into tiers $\mathcal{G}^{(1)}, \ldots, \mathcal{G}^{(K)}$ of user-defined sizes $n_1, \ldots, n_K$, where $\mathcal{G}^{(k)}$ contains the next n_k most confident grasps and the last tier contains all remaining grasps. For $k = 1, 2, \ldots$, we batch-plan from every grasp in $\mathcal{G}^{(k)}$ to all placements in $\mathcal{P}_m$ and collect the feasible trajectories as $\mathcal{S}_k$. The search stops at the first tier with $\mathcal{S}_k \neq \emptyset$, so lower-confidence grasps are tried only when higher-confidence ones fail. Instead of returning the first trajectory found, this returns every feasible trajectory in the successful tier, which keeps planning efficient while still offering diversity in grasps and placements.

The selection step EVALUATE in Algorithm 1 can implement any user-specified criterion [2]. In this work, we execute the trajectory whose final object placement is closest in the XY plane to the target:

$$\tau^* = \arg\min_{\tau \in \mathcal{S}} \big\| \text{xy}\big(T_O(1)\big) - \text{xy}\big(T_p^{\text{des}}\big) \big\|_2, \tag{9}$$

where $\text{xy}(\cdot)$ extracts the planar position of a pose and $T_O(1)$ is the object pose at the end of τ.

IV. EXPERIMENTS

A. Collision Network Evaluation

We evaluate G_ψ on the held-out test split described in Section III-A, which contains 4,874 samples (2,470 positive, 2,404 negative). Training stopped after epoch 64, and we use the checkpoint from epoch 54, which has the lowest validation loss. To approximate sensor noise at inference time, we apply the same coordinate jitter used during training to both the object and environment clouds before subsampling them to 512 points. We report a single seeded noise realization per test sample, and all thresholds are computed from the same set of predicted probabilities.

Table I reports performance for three values of the acceptance threshold δ. Because the network acts as a filter on candidate placements, we report the false acceptance rate (FAR), the fraction of colliding placements that are accepted, and the false rejection rate (FRR), the fraction of collision-free placements that are rejected, alongside accuracy and

Algorithm 1 Pick-and-place planning with collision-free placement sampling

Require: Object cloud Z_O, environment cloud Z_E, number of grasps N_g, start configuration $\boldsymbol{\theta}_0$, placement region $\mathcal{P}$, number of placement candidates N_p, desired placement pose T_p^{des}, acceptance threshold δ, tier sizes $(n_1, \ldots, n_K)$
Ensure: Grasp-to-placement trajectory τ^*, or failure

$(\mathcal{G}, \mathbf{c}) \leftarrow \textsc{GraspGen}(Z_O,\, N_g)$ ▷ N_g candidate grasps T_g^O and confidences
$(\Theta_g, \mathcal{G}_f, \mathbf{c}_f) \leftarrow \textsc{PlanToBatch}(\boldsymbol{\theta}_0,\, \mathcal{G})$ ▷ reachable, collision-free grasps
$\{T_{p,j}\}_{j=1}^{N_p} \leftarrow \textsc{SamplePoses}(\mathcal{P})$ ▷ candidate placements
$\mathbf{Z}_O \leftarrow \left[\, T_{p,1} Z_O,\ \ldots,\ T_{p,N_p} Z_O \,\right]$ ▷ batch of placed object clouds
$\hat{\mathbf{p}} \leftarrow G_\psi(\mathbf{Z}_O,\, Z_E) \in [0,1]^{N_p}$ ▷ collision-free probability of all candidates in one forward pass
$\mathcal{P}_c \leftarrow \{\, T_{p,j} : \hat{p}_j \geq \delta \,\}$ ▷ predicted collision-free placements
Sort $\mathcal{P}_c$ in ascending order of distance to T_p^{des}
for each minibatch $\mathcal{P}_m \subseteq \mathcal{P}_c$, in sorted order **do**
  $\mathcal{S} \leftarrow \textsc{ConfidenceTieredGraspSearch}(\mathcal{P}_m,\, \mathcal{G}_f,\, \Theta_g,\, \mathbf{c}_f,\, Z_O,\, (n_1, \ldots, n_K))$ ▷ candidate plans, Alg. 2
  if $\mathcal{S} \neq \emptyset$ **then**
    return $\tau^* \leftarrow \textsc{Evaluate}(\mathcal{S})$
  end if
end for
return failure

Algorithm 2 Confidence-tiered grasp search

Require: Placement minibatch $\mathcal{P}_m$, feasible grasps $\mathcal{G}_f$ with pick configurations Θ_g and confidences $\mathbf{c}_f$, object cloud Z_O, tier sizes $(n_1, \ldots, n_K)$
Ensure: Set $\mathcal{S}$ of feasible grasp-to-placement trajectories (possibly empty)

Sort $(\mathcal{G}_f, \Theta_g)$ by $\mathbf{c}_f$ in descending order
$i \leftarrow 0$
for $k = 1, \ldots, K$ **do**
  $\mathcal{K}_k \leftarrow$ grasps ranked $i{+}1, \ldots, i{+}n_k$ ▷ n_K: all remaining grasps
  $\mathcal{S} \leftarrow \emptyset$
  for each $(T_g^O, \boldsymbol{\theta}_g) \in \mathcal{K}_k$ **do**
    Attach Z_O to the robot at pose T_g^O ▷ object becomes robot collision geometry
    $\mathcal{S}_g \leftarrow \textsc{BatchPlan}\big(\boldsymbol{\theta}_g,\ \{T_p\, T_g^O : T_p \in \mathcal{P}_m\}\big)$ ▷ one batch over all placements
    Detach Z_O
    $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_g$
  end for
  if $\mathcal{S} \neq \emptyset$ **then**
    return $\mathcal{S}$ ▷ stop at the first tier with a feasible plan
  end if
  $i \leftarrow i + n_k$
end for
return $\emptyset$

precision. Raising δ makes the filter more conservative: from $\delta = 0.5$ to $\delta = 0.7$, FAR drops by roughly half (12.73% to 6.61%) while recall falls from 86.80% to 79.31%, and accuracy changes by less than one percentage point. Accuracy alone therefore hides the trade-off that matters for planning. We use $\delta = 0.6$ in all planning experiments as a middle ground. The network is not the only safeguard against colliding placements. Because collision avoidance is also a constraint of the planning problem (3g), the planner rejects an accepted placement that collides with the observed geometry in Z_E. It cannot detect collisions with occluded geometry, which is the gap the network fills, but it does catch many false acceptances. A false acceptance therefore often costs only planning time, while a false rejection permanently removes a valid placement from consideration. For this reason we do not use the more conservative $\delta = 0.7$.

Because the split holds out individual samples rather than entire clutter configurations or objects, these results do not show that the network generalizes to new scenes or objects. We leave evaluation on held-out scenes and objects to future work.

B. Hardware Pick and Place Experiments

We evaluate Algorithm 1 on pick and place tasks in cluttered tabletop environments, similar to the experimental setup in [1], on a physical KUKA iiwa 7-DOF arm with a Robotiq-2F140 gripper and a RealSense D455 depth camera. Grasp generation, network inference, and motion planning

TABLE I

HELD-OUT TEST PERFORMANCE OF THE COLLISION NETWORK FOR DIFFERENT ACCEPTANCE THRESHOLDS δ.

δ	Accuracy	Precision	Recall	FAR	FRR
0.5	87.03%	87.51%	86.80%	12.73%	13.20%
0.6	86.75%	90.04%	83.04%	9.44%	16.96%
0.7	86.25%	92.49%	79.31%	6.61%	20.69%

TABLE II

HARDWARE EXPERIMENTS: TIMING AND SUCCESS PER RUN, GROUPED BY PICK OBJECT.

Pick Object	Time (s)					Success	
	Grasp generation[a]	Pick planning	Placement sampling	Network inference	Place planning	Grasp	Place
Mustard Bottle	0.068	13.298	0.014	0.500	3.325	Yes	No
	0.063	10.816	0.012	0.484	13.224	Yes	Yes
	0.089	9.899	0.012	0.498	2.241	Yes	No
	0.079	12.232	0.013	0.494	3.075	Yes	Yes
Bleach Cleanser	0.076	9.844	0.012	0.491	5.080	Yes	Yes
	0.073	11.036	0.012	0.494	11.204	Yes	Yes
	0.067	9.498	0.013	0.479	1.761	Yes	Yes
	0.088	7.902	0.013	0.501	3.486	Yes	No

[a]Time to generate 100 candidate grasps per run.

run on a PC with an 8-core AMD Ryzen 7 3700X CPU, NVIDIA GeForce RTX 3090 Ti GPU, and 64 GB RAM. The collision network is trained only on simulated data, with coordinate noise added to the point clouds (Section III-A), and the planner's collision world is built from the partial-view point cloud Z_E observed by the camera. These experiments therefore test whether the network and planner transfer from simulation to real sensor data.

The robot is tasked with picking an object and placing it as close as possible to a desired placement location T_p^{des} on the table. T_p^{des} is the table corner marked by the pink sticker in Fig. 3. We test two cluttered tabletop configurations, shown in Fig. 3. In configuration 1 the robot picks the mustard bottle, and in configuration 2 it picks the bleach cleanser. The pick and clutter objects are the five objects used to train the collision network, since these experiments are intended as a proof of concept of how the network is used within the planning system, not as a test of generalization to unseen objects. We use the same definition of successful grasp and place as [1]: a grasp is successful if the robot lifts the object without dropping it, and a placement is successful if the robot places the grasped object on the placement surface and all objects remain upright.

We run four trials per configuration. For each trial, we generate $N_g = 100$ grasps for the object point cloud and use a motion planning batch size of $|\mathcal{P}_m| = 8$. For the confidence-tiered grasp search, we plan with the top 3 ranked grasps first, followed by the next best 5, and then any remaining grasps if needed, i.e., $(n_1, n_2, n_3) = (3, 5, \text{all remaining})$. We sample $N_p = 100$ random (x, y, yaw) object placements that are used for network inference, with a different random number generator seed for each trial, and accept placements with $\delta = 0.6$. Because we do not have a consistent reference frame for measuring the final object position on hardware, we do not report placement error. Table II reports the recorded metrics for each run, and Fig. 1 shows an example of a successful trial.



Fig. 3. Hardware environment configurations. Configuration 1 (left) uses the mustard bottle as the pick object, and configuration 2 (right) uses the bleach cleanser. The pink sticker on the table corner marks the desired placement location T_p^{des}.

The robot grasped the object in all eight trials and placed it successfully in five. All three placement failures occurred when the robot collided with an object in the environment while executing the placement trajectory. Fig. 4 shows one of these failures, from the third mustard bottle trial. The planner found a collision-free trajectory, but it relies on the assumption from our problem formulation that the hand holds the object rigidly at the planned grasp. During execution, the bottle shifted within the gripper as the fingers made contact (grasp slip), so the held object no longer matched the geometry used for collision checking, and the robot knocked over the bleach cleanser while executing the placement trajectory. The bottle then moved again as the robot released it (place slip), which did not cause a further collision in this trial but can change the object's final pose. Grasp slip and place slip are execution-time behaviors that

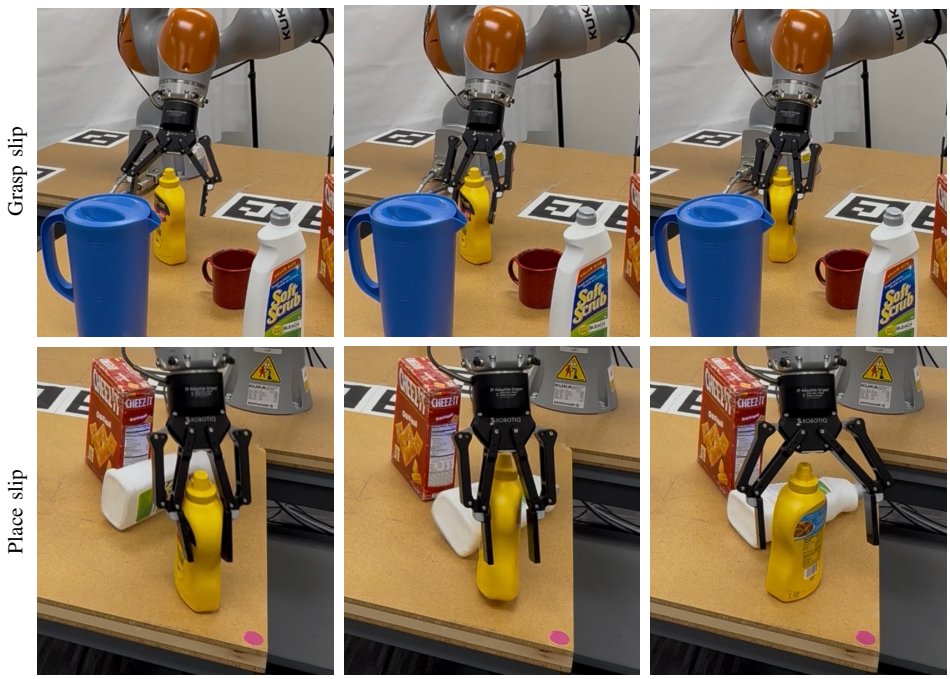


Fig. 4. Execution-time effects that are not modeled during planning, from the third mustard bottle trial in configuration 1. Each row runs left to right in time. Top (grasp slip): the mustard bottle shifts within the gripper fingers as the grasp makes contact. The planner assumes the object is held rigidly at the planned grasp, so this shift is not accounted for, and the robot knocks over the bleach cleanser while executing the placement trajectory. Bottom (place slip): the bottle moves again as the robot releases it. The bleach cleanser is still moving from the earlier collision; no additional collision occurs at release.

our planner does not model, so solving problem (3) does not guarantee a successful placement on hardware. While this work focuses on planning, we intend to address grasp stability and trajectory tracking in future work.

By leveraging batch planning, using a neural network, and reasoning over grasp confidence and network predictions, we see a significant performance increase with our approach in comparison to the approach in [1]. **Our pipeline has an average runtime of 16.57 s with a standard deviation of 4.84 s, while the joint constrained optimization has an average runtime of 58.21 s with a standard deviation of 26.83 s.** The bulk of our pipeline is spent on pick planning (10.57 s on average), which is expected since we are given 100 candidate grasps from the grasp generator. Once we know which grasps can be planned to for picking the object, using only those grasps with the confidence-tiered grasp search makes place planning faster than pick planning in most trials, even in clutter. Place planning time ranges from 1.76 s to 13.22 s. The two longest place planning times (11.20 s and 13.22 s) were due to being unable to find a placement trajectory from the top-ranked tiers. We suspect this is due to sampled placement poses being in tight clutter for the given trial's sampling seed. Network inference takes about 0.5 s per trial, so it is not a bottleneck in our approach.

A successful pick and place does not guarantee that the object ends up close to T_p^{des}. In the first bleach cleanser trial (Fig. 5), both the grasp and the placement succeeded, but the object was placed beside the cracker box, far from the target corner. Our placement candidates are sampled uniformly over the entire table, and Algorithm 1 only sorts them by distance to T_p^{des} after sampling. When few of the uniform samples fall near T_p^{des}, or those that do are rejected by the network or the planner, the closest feasible placement can still be far from the target, which leads to suboptimal placement error. In future work, we plan to replace this simple uniform sampling with a more intelligent placement candidate generation module.

V. CONCLUSION

We presented a collision-conditioned pick and place planning framework that combines a point cloud collision network with a grasp generator and a vectorized motion planner to reason jointly over many grasp and placement pairs. In hardware experiments, it is roughly 3.5× faster than a joint constrained optimization baseline [1], and it transfers from simulated training data to real partial-view point clouds. In future work, we plan to extend the network from collision to stability reasoning, so that it also predicts whether the object remains stable after release [3]–[6]. We also plan to train the network on more diverse objects and environments so that it generalizes to objects not seen during training, to sample grasps more intelligently to improve pick planning times, and to replace uniform placement sampling with a more intelligent placement candidate generation module to reduce placement error. We also plan to handle imperfect execution, namely grasp stability and trajectory tracking, as this work focuses on planning. We believe this combination of learned placement reasoning and vectorized planning is a practical step toward a unified framework for placement reasoning and motion planning in clutter.

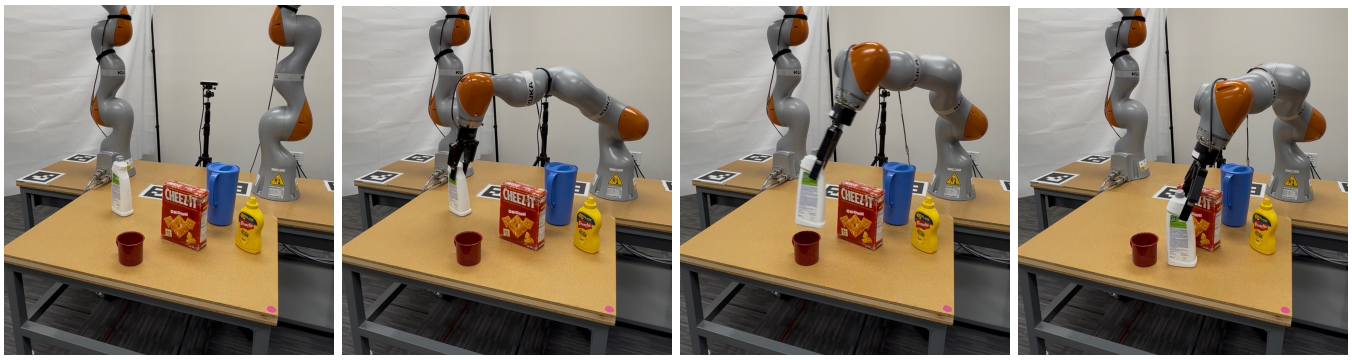


Fig. 5. A successful pick and place with large placement error, from the first bleach cleanser trial in configuration 2. From left to right, the robot picks the bleach cleanser and places it upright beside the cracker box, far from the desired placement location T_p^{des} at the table corner marked by the pink sticker. Placement candidates are sampled uniformly over the table, so the closest candidate that passes collision filtering and planning can still be far from T_p^{des}.

REFERENCES

[1] M. Devendran Shanthi and T. Hermans, "Pick and place planning is better than pick planning then place planning," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2790–2797, 2024.

[2] M. Matak, M. Devendran Shanthi, K. Van Wyk, and T. Hermans, "First plan then evaluate: Multi-target planning with post-planning success evaluation improves learning-based grasping pipelines," *IEEE Robotics and Automation Letters*, 2026.

[3] P. Nadeau and J. Kelly, "Stable object placement planning from contact point robustness," *IEEE Transactions on Robotics*, vol. 41, pp. 3669–3683, 2025. [Online]. Available: http://dx.doi.org/10.1109/TRO.2025.3577049

[4] P. Nadeau, M. Rogel, I. Bilić, I. Petrović, and J. Kelly, "Generating stable placements via physics-guided diffusion models," 2025. [Online]. Available: https://arxiv.org/abs/2509.21664

[5] S. Noh, R. Kang, T. Kim, S. Back, S. Bak, and K. Lee, "Learning to place unseen objects stably using a large-scale simulation," 2023. [Online]. Available: https://arxiv.org/abs/2303.08387

[6] B. Wingender, N. Dengler, R. Menon, S. Pan, and M. Bennewitz, "Robust placeability estimation for model-free unified pick-and-place reasoning," in *The Workshop on Manipulation Robustness: Towards Human-Level Robustness under Real-World Challenges (ICRA 2026)*, 2026. [Online]. Available: https://openreview.net/forum?id=fcr2ug3kTh

[7] M. Danielczuk, A. Mousavian, C. Eppner, and D. Fox, "Object rearrangement using learned implicit collision functions," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[8] A. Murali, A. Mousavian, C. Eppner, A. Fishman, and D. Fox, "CabiNet: Scaling neural collision detection for object rearrangement with procedural scene generation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[9] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer v3: Simpler, faster, stronger," 2024. [Online]. Available: https://arxiv.org/abs/2312.10035

[10] NVIDIA, "Isaac Sim," https://github.com/isaac-sim/IsaacSim, version 5.1.0.

[11] B. Calli, A. Singh, A. Walsman, S. Srinivasa, P. Abbeel, and A. M. Dollar, "The ycb object and model set: Towards common benchmarks for manipulation research," in *2015 International Conference on Advanced Robotics (ICAR)*, 2015, pp. 510–517.

[12] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[14] A. Murali, B. Sundaralingam, Y.-W. Chao, J. Yamada, W. Yuan, M. Carlson, F. Ramos, S. Birchfield, D. Fox, and C. Eppner, "Graspgen: A diffusion-based framework for 6-dof grasping with on-generator training," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2026. [Online]. Available: https://arxiv.org/abs/2507.13097

[15] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. V. Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, N. Ratliff, and D. Fox, "curobo: Parallelized collision-free minimum-jerk robot motion generation," 2023.